

Generátor riešených príkladov sústav lineárnych rovníc

Svetlana Pivarčiová

Ústav informatiky, informačných systémov a softvérového inžinierstva

Fakulta informatiky a informačných technológií

Slovenská technická univerzita v Bratislave

Bratislava, Slovensko

xpivarciovas@stuba.sk

Abstrakt—Táto práca opisuje návrh a implementáciu algoritmického generátora riešených úloh zo sústav lineárnych rovníc. Cieľom riešenia je automaticky vytvárať matematicky správne, numericky kontrolované a didakticky orientované príklady vo Wolfram Language. Na rozdiel od bežných riešičov, ktoré spracúvajú už zadanú sústavu, navrhnutý systém konštruuje zadanie, typ riešenia a krokový postup ako jeden kontrolovaný celok.

Hlavná verzia generátora je implementovaná v balíku `MatrixGenerator.wl` pre prostredie Wolfram Mathematica. Súčasťou riešenia je aj verzia pre prostredie Mathio, ktorá zachováva rovnakú generovacie logiku, ale prispôsobuje výstupy webovému prostrediu. Systém zahŕňa desať generátorov pokrývajúcich maticové aj rovnicové metódy riešenia, napríklad Gaussovu a Gaussovu–Jordanovu elimináciu, LU rozklad, Choleského rozklad, Cramerovo pravidlo, eliminačnú metódu a dosadzovaciu metódu.

Pri generovaní sa kontrolujú číselné hranice, vhodnosť kontrolovaných medzivýsledkov a správnosť deklarovaného typu riešenia. Výsledkom je rozšíriteľný nástroj na tvorbu riešených príkladov, ktorý môže slúžiť pri príprave výučbových materiálov a samostatnom precvičovaní sústav lineárnych rovníc.

KLúčové slová—sústavy lineárnych rovníc, automatické generovanie úloh, Wolfram Language, Wolfram Mathematica, didaktické príklady

LAICKÝ ABSTRAKT

Práca opisuje program, ktorý automaticky vytvára riešené príklady zo sústav lineárnych rovníc. Cieľom nie je vytvoriť náhodnú úlohu, ale úlohu vhodnú na učenie – s primeranými číslami, jasným postupom riešenia a správnym výsledkom.

Program pracuje inak ako bežná kalkulačka. Najprv si pripraví riešenie alebo vnútorný tvar úlohy, pri ktorom je už známe, aký typ riešenia má vzniknúť. Až potom z tejto štruktúry vytvorí zadanie pre študenta. Vďaka tomu môže kontrolovať, aby úloha neobsahovala zbytočne veľké čísla alebo neprehľadné medzivýpočty.

Pri maticových metódach program pracuje s maticami, riadkovými úpravami, rozkladmi alebo determinantmi. Pri rovnicových metódach pripravuje sústavy tak, aby sa dali riešiť prirodzeným eliminačným alebo dosadzovacím postupom. Hlavná verzia je určená pre prostredie Wolfram Mathematica. Doplnková verzia pre webové prostredie Mathio umožňuje používať riešené príklady aj bez lokálnej inštalácie Mathematica.

Výsledkom je nástroj, ktorý môže pomôcť učiteľom pri príprave riešených príkladov a študentom pri samostatnom precvičovaní.

I. ÚVOD

Sústavy lineárnych rovníc patria k základným okruhom algebry a lineárnej algebry. Študenti sa s nimi stretávajú pri riešení jednoduchých rovnicových sústav aj pri práci s maticami, riadkovými úpravami, determinantmi a rozkladmi matic. Táto oblasť je vhodná aj na rozvíjanie algoritmického myslenia, pretože jednotlivé metódy pozostávajú z presne určených krokov. Pri výučbe však nestačí ovládať samotný algoritmus; dôležité je aj porozumenie vzťahom medzi rovnicovým zápisom, maticovou reprezentáciou a interpretáciou riešenia [1], [2].

Pri príprave úloh nestačí zabezpečiť iba matematickú správnosť. Dve sústavy s rovnakým počtom rovníc a neznámych sa môžu výrazne líšiť praktickou náročnosťou, napríklad počtom krokov, charakterom operácií, veľkosťou koeficientov alebo výskytom zlomkov. Tieto faktory môžu zvyšovať kognitívnu záťaž riešiteľa [3] a pri práci so zlomkami sa môže prejaviť aj jav označovaný ako *natural number bias* [4], [5]. Pri generovaní riešených príkladov je preto potrebné kontrolovať nielen výsledok, ale aj podobu zadania, numerickú náročnosť medzivýpočtov a vhodnosť úlohy pre zvolenú metódu.

Navrhované riešenie nevychádza z priameho náhodného výberu koeficientov. Generátor najprv pripraví štruktúru podľa požadovaného typu riešenia alebo metódy a až z nej zostaví konkrétnu sústavu. Tento prístup nadväzuje na automatizované generovanie úloh založené na explicitných modeloch položiek, parametroch a kontrolných pravidlách [6], [7].

Hlavná implementácia v balíku `MatrixGenerator.wl` obsahuje generátory pre maticové aj rovnicové metódy. Maticové generátory vychádzajú zo sústavy v tvare $Ax = b$, pričom pri konštrukcii alebo riešení môžu používať aj augmentovanú maticu. Rovnicové generátory pracujú priamo s rovnicami a sú určené pre eliminačnú a dosadzovaciu metódu. Doplnková verzia pre prostredie Mathio používa rovnakú generovacie logiku, ale výstupy prispôsobuje webovému rozhraniu.

Cieľom tejto práce je formalizovať princíp návrhu takéhoto generátora. Generovaný príklad sa chápe ako algoritmicky zostavený objekt, pri ktorom sú súčasne kontrolované štyri vlastnosti: typ riešenia, vhodnosť pre zvolenú metódu, numerická náročnosť a forma krokového riešenia. Takéto vymedzenie odlišuje navrhnutý systém od riešiča existujúcich sústav aj od jednoduchého náhodného generátora koeficientov.

II. METÓDY

A. Všeobecná architektúra generovania

Implementované generátory majú jednotný parametrický model. Každý generovaný príklad je určený voľbou generátora G , obtiažnosti d , režimu výstupu m , typu riešenia t a doplnkových volieb o . Generovanie jednej úlohy možno zapísať ako

$$I = F(G, d, m, t, o), \quad (1)$$

kde F označuje konštrukčný algoritmus príslušného generátora, I výsledný riešený príklad a o predstavuje doplnkové voľby podporované iba niektorými generátormi.

Parameter `diff` určuje úroveň obtiažnosti a môže nadobúdať hodnoty "EASY", "MEDIUM" alebo "HARD". Parameter `mode` určuje rozsah výstupu, teda či sa zobrazí iba zadanie, zadanie s výsledkom alebo kompletný krokový postup. Voľba `SolutionType` sa používa pri generátoroch, ktoré podporujú viac typov riešenia, a určuje, či má mať sústava jedno, žiadne alebo nekonečne veľa riešení.

Doplnkové možnosti sa líšia v závislosti od konkrétneho generátora. Pri maticových generátoroch je možné využiť parameter `TaskFormat`, ktorý definuje spôsob zobrazenia zadania. Hodnota "MATRIX" zobrazí úlohu v maticovom tvare, zatiaľ čo hodnota "EQUATIONS" zobrazí ekvivalentný rovnícový zápis. Pri rovnícových generátoroch `GenElimination` a `GenSubstitution` možno použiť parameter `Visualization`, ktorý určuje, či sa má k riešeniu pridať geometrická vizualizácia.

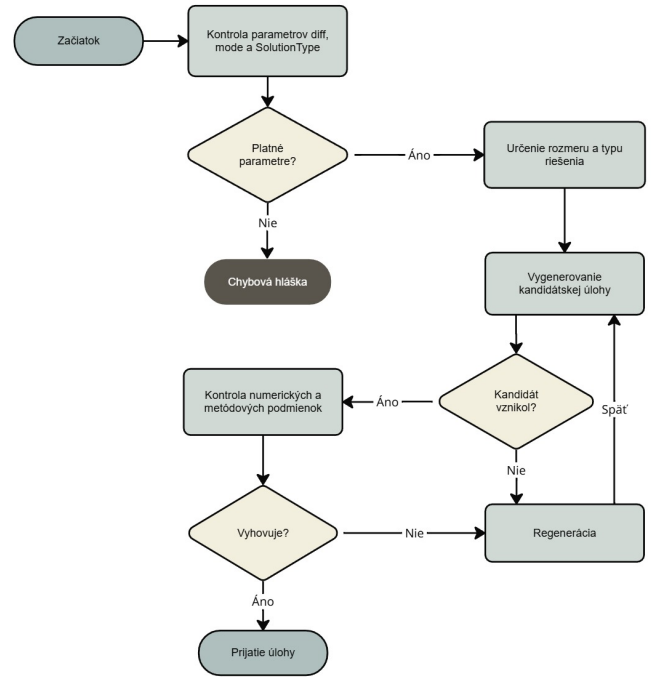
Základom návrhu je opačný postup než pri bežnom náhodnom generovaní: najprv sa pripraví vhodné jadro úlohy a až z neho sa vytvorí zadanie. Generátor sa nespolieha na to, že náhodne vytvorená sústava bude mať vhodné vlastnosti. Namiesto toho si najprv pripraví riešenie, vyriešenú maticu, vhodný rozklad alebo menšiu vnútornú sústavu. Týmto spôsobom vznikne zadanie, pri ktorom sú hlavné vlastnosti úlohy známe ešte pred jej zobrazením študentovi.

Všeobecný postup generovania je založený na opakovanom vytváraní kandidátskej úlohy. Generátor zostaví vnútorné jadro, vytvorí z neho zadanie a overí, či kandidát vyhovuje požiadavkám konkrétnej metódy. Ak kontrola zlyhá, kandidát sa zamietne a proces sa zopakuje. Tento proces je znázornený na obrázku 1.

Konkrétne konštrukčné stratégie sa líšia podľa typu generátora. Maticové generátory vychádzajú zo sústavy $Ax = b$, ale pri vnútornom spracovaní využívajú podľa metódy aj augmentované matice, rozklady alebo determinanty. Rovnicové generátory vytvárajú sústavy pripravené na eliminačný alebo dosadzovací postup. Tieto rozdiely sú zhrnuté v nasledujúcej podsekcii.

B. Konštrukcia úloh podľa typu generátora

Jednotlivé generátory sa líšia najmä tým, akú vnútornú štruktúru používajú pri tvorbe zadania a aký riešiteľský postup majú demonštrovať. Generátory `GenGauss`, `GenGaussJordan` a `GenGaussJordanPivot` vychádzajú z kontrolovanej augmentovanej matice. Pri sústavách s jedným



Obr. 1. Všeobecný proces prijatia alebo zamietnutia kandidátskej úlohy.

riešením sa generovanie začína z tvaru $(I \mid x)$, pri sústavách bez riešenia zo štruktúry so sporovým riadkom a pri sústavách s nekonečne veľa riešeniami zo štruktúry s voľnými premennými. Následnými riadkovými úpravami sa z tejto vnútornej reprezentácie vytvorí menej zjavné zadanie.

Generátory `GenElemGJ` a `GenInverse` používajú podobný princíp, ale sú určené iba pre sústavy s jedným riešením. Pri elementárnych maticiach sa jednotlivé riadkové úpravy zapisujú maticami E_i , zatiaľ čo pri inverznej metóde sa riešenie získava výpočtom A^{-1} z rozšírenej matice $(A \mid I)$.

Generátory `GenLU`, `GenCholesky` a `GenCramer` zostavujú zadanie podľa požiadaviek príslušnej metódy. Pri LU rozklade sa matica vytvára zo súčinu $A = LU$, pri Choleského rozklade z tvaru $A = LL^T$ a pri Cramerovom pravidle sa kontroluje vhodnosť determinantov. Vo všetkých prípadoch sa najprv zvolí riešenie x , pravá strana sa dopočíta ako $b = Ax$ a výsledná sústava prirodzene vedie k precvičovanému postupu.

Rovnicové generátory `GenElimination` a `GenSubstitution` pracujú priamo s rovnicami. Pri jednoduchých úlohách používajú sústavy 2×2 , pri náročnejších 3×3 . Zadanía sú konštruované tak, aby podporovali prirodzený eliminačný alebo dosadzovací postup, a v podporovaných prípadoch môžu byť doplnené geometrickou vizualizáciou prepájajúcou algebraický postup s geometrickou interpretáciou riešenia.

Všetky generátory preto sledujú, či čísla v hlavných častiach zadania, riešenia a medzivýpočtov zostávajú v prijateľnom rozsahu. Matematicky správna sústava sa prijme až vtedy, keď má požadovaný typ riešenia a jej kontrolované koeficienty aj medzivýpočty zostávajú primerané pre riešený príklad.

C. Obtiažnosť a podporované typy úloh

Obtiažnosť v navrhnutom systéme nie je definovaná iba rozmerom sústavy. Pri jednotlivých generátoroch ju ovplyvňuje aj štruktúra zadania, počet potrebných úprav, výskyt pivotovania, potreba normalizácie, tvar rovníc a rozsah kontrolovaných medzivýsledkov. Takéto chápanie obtiažnosti zodpovedá pohľadu kognitívnej záťaž, podľa ktorého náročnosť lineárnych rovníc závisí aj od počtu a charakteru operácií, ktoré musí riešiť počas riešenia sledovať [3]. Cieľom preto nie je vytvárať náročnosť pomocou veľkých čísel, ale pomocou postupov, ktoré sú charakteristické pre danú metódu.

Rozmery a podporované typy riešenia závisia od konkrétneho generátora. Maticové generátory používajú rozmery 3×3 až 6×6 . Rovnicové generátory majú pre jednoduché úlohy rozmer 2×2 , pre stredné a náročné úlohy 3×3 . Prehľad rozmerov podľa obtiažnosti je uvedený v tabuľke I.

Tabuľka I
ROZMERY GENERÁTOROV PODĽA OBTIAŽNOSTI.

Generátor	EASY	MEDIUM	HARD
Maticové generátory			
GenGauss	3×3	5×5	6×6
GenGaussJordan	3×3	5×5	6×6
GenGaussJordanPivot	3×3	5×5	6×6
GenElemGJ	3×3	4×4	5×5
GenCramer	3×3	4×4	5×5
GenInverse	3×3	4×4	6×6
GenLU	3×3	4×4	6×6
GenCholesky	3×3	4×4	6×6
Rovnicové generátory			
GenElimination	2×2	3×3	3×3
GenSubstitution	2×2	3×3	3×3

Nie všetky generátory podporujú všetky typy riešenia. Typy "NONE" a "INFINITE" sú zahrnuté pri Gaussových metódach a rovnicových generátoroch, pretože práve pri týchto postupoch sa v školských a testových úlohách najčastejšie pracuje aj so sústavami bez riešenia alebo s nekonečne veľa riešeniami. Ostatné generátory sú zamerané na typ "ONE", keďže ich cieľom je precvičiť konkrétny výpočtový postup pri jednoznačne riešiteľnej sústave.

Tabuľka II
PODPOROVANÉ TYPY RIEŠENIA JEDNOTLIVÝCH GENERÁTOROV.

Generátor	Podporované typy riešenia
Maticové generátory	
GenGauss	"ONE", "NONE", "INFINITE"
GenGaussJordan	"ONE", "NONE", "INFINITE"
GenGaussJordanPivot	"ONE", "NONE", "INFINITE"
GenElemGJ	"ONE"
GenCramer	"ONE"
GenInverse	"ONE"
GenLU	"ONE"
GenCholesky	"ONE"
Rovnicové generátory	
GenElimination	"ONE", "NONE", "INFINITE"
GenSubstitution	"ONE", "NONE", "INFINITE"

D. Celočíselné riadkové úpravy

Pri návrhu bolo potrebné obmedziť najmä veľkosť čísel a neprehľadnosť medzivýpočtov. Generátor sa má podľa možnosti vyhýbať veľmi veľkým hodnotám a aritmeticky nevhodným krokom. Cieľom je, aby sa študent sústredil na metódu riešenia, nie na zbytočne náročnú aritmetiku. Toto rozhodnutie je motivované aj tým, že práca so zlomkami môže predstavovať samostatný zdroj ťažkostí, ak nie je hlavným didaktickým cieľom riešenej úlohy [4].

Pri eliminačných krokoch sa preto používa celočíselná forma riadkových úprav. Numerická kontrola je v implementácii rozdelená na dve úrovne. Maximálna hranica `$MaxBounds` predstavuje tvrdé obmedzenie pre prijatie hotového príkladu. Pracovná hranica `$Bounds` sa používa počas konštrukcie kandidátskej úlohy, napríklad pri výbere koeficientov, miešani riadkov a kontrole nových medzivýsledkov. Kandidát sa prijme iba vtedy, ak kontrolované numerické hodnoty zostanú v povolenom rozsahu. Tento postup umožňuje odmietat matematicky správne, ale didakticky nevhodné úlohy.

V základnom nastavení bola použitá hodnota `$MaxBounds` rovná 20. Táto hodnota predstavuje kompromis medzi didaktickou prehľadnosťou a dostatočnou variabilitou úloh. Príliš malé hranice by obmedzovali generátor na veľmi jednoduché koeficienty a zároveň by výrazne zvyšovali počet zamietnutých kandidátov. Naopak, príliš veľké hranice by znižovali počet opakovaných pokusov, ale mohli by viesť k výpočtovo neprehľadným príkladom. Hodnota 20 umožňuje zachovať menšie hodnoty v normalizovaných zadaniach a zároveň pripustiť netriviálne, ale stále zvládnuteľné hodnoty v medzivýpočtoch.

Pracovná hranica sa nevolí ako nezávislá pevná konštanta, ale odvodzuje sa od maximálnej hranice. Nech M označuje maximálnu hranicu a B pracovnú hranicu. Potom sa pracovná hranica určí vzťahom

$$B = 4 + \left\lfloor \frac{M}{1.4 + 0.156\sqrt{M}} \right\rfloor. \quad (2)$$

Pri nastavení $M = 20$ vychádza $B = 13$. Takýto spôsob nastavenia znižuje potrebu manuálnej kalibrácie pri zmene maximálnej hranice a umožňuje použiť rovnaký princíp pri generátoroch s rozdielnym charakterom operácií. Pri niektorých generátoroch rastú hodnoty najmä sčítaním alebo odčítaním riadkov, pri iných vznikajú väčšie čísla ako dôsledok násobenia, rozkladov alebo determinantov. Jedna pevná ručne nastavená pracovná hranica preto nemusí byť rovnako vhodná pre všetky generátory.

Pri rovnicových generátoroch môže byť zadanie zámerne zobrazené aj v menej upravenom tvare, ktorý predchádza normalizácii rovníc. V takomto zápise sa môžu objaviť väčšie, spravidla dvojciferné hodnoty než v normalizovanom tvare sústavy, pretože úlohou riešiteľa je najprv presunúť členy a následne rovnice zjednodušiť vydelením spoločným deliteľom. Po tejto úprave je normalizovaný tvar sústavy opäť kontrolovaný podľa nastavených numerických hraníc. Tieto hodnoty preto predstavujú súčasť didaktického postupu a nie porušenie kontroly jadra príkladu.

Ak sa má v riadku R_r vynulovať prvok a pomocou pivotu p v riadku R_i , implementácia vypočíta

$$g = \gcd(p, a), \quad (3)$$

a následne použije hodnoty

$$p_2 = \frac{p}{g}, \quad a_2 = \frac{a}{g}. \quad (4)$$

Nový riadok sa vytvorí ako

$$R_r \leftarrow p_2 R_r - a_2 R_i. \quad (5)$$

Po operácii sa riadok môže ešte skrátiť spoločným deliteľom. Tým sa obmedzuje rast koeficientov a zároveň sa predchádza zavádzaniu zlomkov.

Tento princíp sa používa pri generovaní aj pri samotných riešiteľských krokoch. Z pohľadu študenta ide stále o bežnú elimináciu, ale implementácia priebežne kontroluje, aby medzivýsledky zostali numericky prehľadné.

E. Validácia generovaných sústav

Každý prijatý príklad musí prejsť validačnými kontrolami. Generátor sa preto nespolieha iba na to, že príklad vznikol konštrukčným postupom. Po zostavení sa kandidát ešte raz overí nezávisle od spôsobu, ktorým vznikol.

Pri sústavách s jedným riešením sa správnosť výsledku overuje dosadením do pôvodnej sústavy. Ak má sústava tvar $Ax = b$, kontroluje sa, či vypočítaný vektor x skutočne spĺňa pôvodné rovnice. Pri sústavách bez riešenia alebo s nekonečne veľa riešeniami sa používa porovnanie hodnôt matice koeficientov A a rozšírenej matice $[A | b]$. Ak platí

$$\text{rank}(A) \neq \text{rank}([A | b]), \quad (6)$$

sústava je nekonzistentná a nemá riešenie. Ak pre počet neznámych n platí

$$\text{rank}(A) = \text{rank}([A | b]) = n, \quad (7)$$

sústava má práve jedno riešenie. Ak platí

$$\text{rank}(A) = \text{rank}([A | b]) < n, \quad (8)$$

sústava má nekonečne veľa riešení.

Okrem všeobecnej kontroly typu riešenia obsahujú niektoré generátory aj metódovo špecifickú validáciu. Pri LU rozklade sa overuje, či rozklad prebehne bez nulového pivotu a či následné riešenie pomocných sústav vedie k správne výsledku. Pri Choleského rozklade sa kontroluje symetria matice, vhodnosť kladne definitnej štruktúry a celočíselnosť výpočtu. Pri Cramerovom pravidle sa kontroluje nenulovosť hlavného determinantu a vhodnosť determinantov pomocných matic A_i . Pri metóde inverznej matice sa overuje vypočítaná inverzná matica aj výsledné riešenie sústavy.

Validačné kontroly majú dve úlohy. Prvou je zabrániť prijatiu matematicky nesprávneho príkladu. Druhou je zabezpečiť, aby prijatý príklad zodpovedal didaktickému zámeru generátora. Kandidát sa teda môže odmietnuť aj v prípade, že je matematicky správny, ale nehodí sa pre zvolený postup alebo prekračuje nastavené číselné hranice. V kontexte

automatického generovania úloh je takáto kontrola dôležitá, pretože formálne podobné položky nemusia byť automaticky porovnateľné z hľadiska kvality alebo náročnosti [7].

III. VÝSLEDKY

Výsledkom práce je hlavná implementácia v balíku `MatrixGenerator.wl`, ktorá generuje riešené príklady pre viacero metód riešenia sústav lineárnych rovníc. Balík poskytuje jednotné rozhranie volania, podporuje rôzne úrovne obtiažnosti a umožňuje zobrazit' buď samotné zadanie, zadanie s výsledkom alebo zadanie s krokovým postupom a výsledkom. Súčasťou výsledku je aj verzia pre prostredie Mathio, ktorej kód je uložený v súbore `MatrixGenerator-MATHIO.wl`.

Implementované generátory pokrývajú maticové aj rovnicové prístupy. Maticové generátory zahŕňajú Gaussovu elimináciu, Gaussovu–Jordanovu elimináciu, pivotovanie, elementárne matice, inverznú maticu, LU rozklad, Choleského rozklad a Cramerovo pravidlo. Rovnicové generátory zahŕňajú eliminačnú a dosadzovaciu metódu.

V notebookovej verzii sú výstupy generované ako bunky prostredia Wolfram Mathematica. Kompletne riešenie obsahuje matematický zápis, textové vysvetlenia, medzikroky, výsledok a pri vybraných generátoroch aj zvýraznené časti postupu alebo vizualizáciu. Pri elementárnych maticiach sa zobrazujú aj matice zodpovedajúce jednotlivým riadkovým úpravám. Pri inverznej metóde sa zobrazuje výpočet A^{-1} a následné použitie tejto matice na výpočet riešenia. Pri rovnicových generátoroch môže byť výstup doplnený aj o geometrickú vizualizáciu, ktorá podporuje interpretáciu riešenia pomocou priamok alebo rovín.

Hlavným výsledkom je súbor generátorov, ktoré vytvárajú úlohy podľa vopred určených pravidiel. Každá prijatá úloha je odvodená z pripraveného jadra a následne overená samostatnými kontrolami. Pri Gaussových metódach sa využíva riadkové premiešanie známeho vyriešeného tvaru. Pri LU a Choleského rozklade sa matica vytvára priamo zo štruktúr vhodných pre daný rozklad. Pri Cramerovom pravidle sa kontroluje vhodnosť determinantov. Pri rovnicových metódach sa väčšie sústavy skladajú z menších kontrolovaných jadier.

Takýto prístup umožňuje vytvárať rôzne varianty úloh, ale zároveň udržiavať pod kontrolou typ riešenia, numerickú náročnosť a didaktický postup.

Výsledkom teda nie je len zoznam samostatných funkcií, ale jednotný spôsob, akým sa úlohy konštruujú, kontrolujú a zobrazujú. Každý prijatý príklad má explicitne určenú metódu riešenia, kontrolovaný režim výstupu, overenú matematickú správnosť a obmedzené numerické hodnoty. Pri generátoroch s podporou viacerých typov riešenia je zároveň overené, že výsledná sústava zodpovedá deklarovanej triede "ONE", "NONE" alebo "INFINITE".

A. Technické overenie generovania

Technické overenie bolo zamerané na stabilitu generovania a dodržanie numerických obmedzení. Pri každom teste sa sledovalo, či generátor vytvorí prijatý príklad zodpovedajúci zvolenému typu riešenia, či výstup neskončí zlyhaním a koľko

kandidátskych úloh bolo potrebné vytvoriť pred prijatím výsledku. Cieľom testovania nebolo empirické hodnotenie náročnosti úloh študentmi, ale overenie správania validačných pravidiel a mechanizmu opakovaného generovania kandidátov.

Testovanie bolo rozdelené podľa typu riešenia. Typ "ONE" podporujú všetky implementované generátory, preto tvoril hlavnú časť overenia. Typy "NONE" a "INFINITE" podporujú iba generátory GenGauss, GenGaussJordan, GenGaussJordanPivot, GenElimination a GenSubstitution, preto boli overené samostatne s menším počtom opakovaní.

Tabuľka III
ROZSAH TECHNICKÉHO OVERENIA GENEROVANIA.

Typ riešenia	Generátory	Typy obtiažnosti	Opakovanie	Testov
"ONE"	10	3	10	300
"NONE"	5	3	5	75
"INFINITE"	5	3	5	75
Spolu				450

Stĺpec *Opakovanie* vyjadruje počet opakovaní pre jeden generátor a jednu obtiažnosť pri danom type riešenia.

Pre typ "ONE" bolo vykonaných $10 \times 3 \times 10 = 300$ testov. Pre typy "NONE" a "INFINITE" bolo vykonaných $5 \times 3 \times 5 = 75$ testov pre každý z týchto typov. Rozdielny počet opakovaní bol zvolený preto, aby hlavné vyhodnotenie nebolo neprimerane ovplyvnené generátormi, ktoré podporujú viac typov riešenia.

Tabuľka IV
VÝSLEDKY TECHNICKÉHO OVERENIA GENEROVANIA.

Typ	Spustení	Prijatých	AVG počet pregenerovaní	MAX počet pregenerovaní
"ONE"	300	300	2,71	47
"NONE"	75	75	0,97	8
"INFINITE"	75	75	0,65	22
Spolu	450	450	2,08	47

Všetkých 450 testovacích spustení skončilo prijatím príkladu. V kontrolovaných častiach prijatých výstupov nebolo v testovanej vzorke zaznamenané porušenie nastavených numerických hraníc ani výskyt zlomkov. Priemerný počet pokusov bol najvyšší pri type "ONE", pretože táto skupina zahŕňa všetky generátory vrátane tých, ktoré majú prísnejšie štruktúrne alebo numerické podmienky.

Do kontroly numerických hraníc neboli zahrnuté pomocné výpisy skúšky správnosti parametrizácie pri niektorých prípadoch "INFINITE" v rovnicových generátoroch s rozmerom 3×3 . Táto časť slúži ako doplnkové overenie dosadením parametrického riešenia, nie ako konštrukčná časť zadania ani hlavný riešiteľský postup. Jej zahrnutie do kritéria prijatia by neúmerne zvýšilo počet zamietnutých kandidátov, pretože pri dosádzaní parametrických výrazov môžu dočasne vzniknúť väčšie hodnoty, hoci samotná sústava aj jej riešenie zostávajú didakticky primerané.

IV. DISKUSIA

Implementované riešenie ukazuje, že matematické príklady možno generovať tak, aby náhodnosť slúžila iba na obmenu zadania, nie na určovanie jeho základných vlastností. Náhodnosť sa v systéme používa iba v medziach pravidiel konkrétnej metódy, numerických obmedzení a validačných kontrol. Vďaka tomu generátor nevytvára iba formálne správne zadania, ale aj riešené príklady s predvídateľnejšou štruktúrou a zrozumiteľným postupom.

Silnou stránkou návrhu je prispôbenie jednotlivých generátorov konkrétnym metódam riešenia. Náročnosť príkladu má preto vyplývať najmä z použitej metódy, nie zo zbytočne veľkých čísel alebo neprehľadných medzivýsledkov.

Výsledky technického overenia ukazujú, že opakované generovanie kandidátov je prakticky použiteľné aj pri prísnejších numerických obmedzeniach. Vyšší počet pokusov sa prirodzene objavuje najmä pri generátoroch, ktorých konštrukcia obsahuje viac štruktúrnych podmienok, napríklad pri determinantových výpočtoch, rozkladoch alebo výpočte inverznej matice. Naopak, rovnicové generátory často prijímajú kandidáta už pri prvom pokuse, pretože vychádzajú z menších kontrolovaných jadier.

Jadro generovania nie je založené na trénovanom modeli strojového učenia. Úlohy vznikajú algoritmicke pomocou pravidiel, náhodných výberov a validačných obmedzení. Tento prístup sa líši od neurónových prístupov, ktoré matematické úlohy riešia, vysvetľujú alebo generujú pomocou modelov učenia z dát [8]. Výhodou navrhnutého riešenia je transparentnosť: každý prijatý príklad možno spätne vysvetliť pomocou konštrukčných pravidiel a validačných kontrol.

Jedným z obmedzení riešenia je závislosť od prostredí založených na Wolfram Language. Ten poskytuje nástroje na presné, symbolické aj numerické riešenie lineárnych systémov [9], čo je vhodné pre implementáciu kontrolovaných generátorov sústav lineárnych rovníc. Notebookové prostredie Wolfram Mathematica zároveň umožňuje prehľadne zobrazovať matematický zápis, medzikroky, textové vysvetlenia a doplnkové vizualizácie. Nemusi však byť dostupné každému používateľovi. Toto obmedzenie čiastočne zmierňuje Mathio verzia, ktorá sprístupňuje riešené príklady vo webovom prostredí, hoci s jednoduchšími možnosťami interakcie.

Ďalším obmedzením je nutnosť udržať zadanie aj medzivýsledky pod stanovenými numerickými hranicami. Pri metódach, v ktorých sa násobia matice, počítajú determinanty alebo používajú maticové rozklady, preto musia byť vstupné koeficienty často menšie než pri metódach založených najmä na riadkových úpravách. Ide o kompromis medzi variabilitou zadania a didaktickou prehľadnosťou výpočtu.

Obmedzením je aj to, že obtiažnosť generovaných úloh je v aktuálnej verzii určená konštrukčnými pravidlami, nie empirickým testovaním na študentoch. Technické overenie potvrdzuje stabilitu generovania a dodržanie numerických obmedzení, ale nemeria skutočnú náročnosť úloh podľa času riešenia, chybovosti alebo subjektívneho hodnotenia používateľov. Výsledky testovania preto treba chápať ako overenie implementácie a validačných pravidiel, nie ako používateľskú štúdiu.

Do budúcnosti by bolo vhodné prakticky overiť použiteľnosť generovaných príkladov vo výučbe. Sledovať by sa mohla najmä chybovosť, čas riešenia, vnímaná náročnosť a zrozumiteľnosť jednotlivých krokov. Ďalším smerom rozšírenia je doplnenie nových typov úloh, úprava rozsahu komentárov podľa úrovne používateľa a ďalší rozvoj Mathio verzie tak, aby sa spôsobom prezentácie viac približovala notebookovej verzii.

V. DOPLŇUJÚCE MATERIÁLY A DÁTA

Doplňujúce materiály k práci tvoria súbory `MatrixGenerator.wl`, `Execute.nb` a `MatrixGenerator-MATHIO.wl`. Súbor `MatrixGenerator.wl` obsahuje hlavnú implementáciu generátorov, pomocných funkcií a validačných kontrol pre prostredie Wolfram Mathematica. Notebook `Execute.nb` slúži na načítanie balíka a obsahuje základné ukážky volania jednotlivých generátorov. Súbor `MatrixGenerator-MATHIO.wl` obsahuje verziu kódu pripravenú pre prostredie Mathio, v ktorej sú výstupy prispôbené webovému zobrazeniu.

Na použitie riešenia nie je potrebná externá dátová množina, keďže úlohy sa generujú algoritmicky podľa pravidiel implementovaných v kóde. Typické načítanie hlavného balíka v notebooku má tvar:

```
SetDirectory[NotebookDirectory[]];
$CharacterEncoding = "UTF-8";
<< "MatrixGenerator.wl";
```

Po načítaní balíka je možné spúšťať jednotlivé generátory s požadovanou obtiažnosťou, režimom výstupu a podporovanými voliteľnými parametrami. V prostredí Mathio sa generátor používa cez webovú úlohu, v ktorej používateľ zvolí typ generátora, obtiažnosť a podporované parametre. Podľa zvoleného režimu sa následne zobrazí buď samotné zadanie, zadanie s výsledkom, alebo zadanie s krokovým riešením

a výsledkom. Keďže generovanie obsahuje náhodné výbery, konkrétne zadania sa pri opakovanom spustení môžu líšiť.

Pri tvorbe textových výpisov krokových postupov boli použité nástroje generatívnej umelej inteligencie na návrh formulácií a kontrolu zrozumiteľnosti, nie ako prostriedok na posudzovanie matematickej správnosti generovaných príkladov.

LITERATÚRA

- [1] R. J. Rensaa, N. M. Hogstad, and J. Monaghan, "Perspectives and reflections on teaching linear algebra," *Teaching Mathematics and its Applications: An International Journal of the IMA*, vol. 39, no. 4, pp. 296–309, 2020.
- [2] M. A. Tashtoush, Y. Wardat, M. M. Al-Shannaq, R. AlAli, S. Saleh, and K. AL-Saud, "Conceptual understanding for systems of linear equations: Difficulties and challenges," *Information Sciences Letters*, vol. 12, no. 12, pp. 2491–2503, 2023. [Online]. Available: <https://www.naturalspublishing.com/files/published/hd967ovy58uy52.pdf>
- [3] B. H. Ngu and H. P. Phan, "Advancing the study of solving linear equations with negative pronumerals: A smarter way from a cognitive load perspective," *PLOS ONE*, vol. 17, no. 3, p. e0265547, 2022. [Online]. Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0265547>
- [4] F. Reinhold, A. Obersteiner, S. Hoch, S. I. Hofer, and K. Reiss, "The interplay between the natural number bias and fraction magnitude processing in low-achieving students," *Frontiers in Education*, vol. 5, p. 29, 2020. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/educ.2020.00029/full>
- [5] A. Obersteiner, M. W. Alibali, and V. Marupudi, "Complex fraction comparisons and the natural number bias: The role of benchmarks," *Learning and Instruction*, vol. 67, p. 101307, 2020.
- [6] M. J. Gierl and H. Lai, "The role of item models in automatic item generation," *International Journal of Testing*, vol. 12, no. 3, pp. 273–298, 2012.
- [7] Y. Fu, E. M. Choe, H. Lim, and J. Choi, "An evaluation of automatic item generation: A case study of weak theory approach," *Educational Measurement: Issues and Practice*, vol. 41, no. 4, pp. 10–22, 2022.
- [8] I. Drori, S. Zhang, R. Shuttlesworth, L. Tang, A. Lu, E. Ke, K. Liu, L. Chen, S. Tran, N. Cheng, R. Wang, N. Singh, T. L. Patti, J. Lynch, A. Shporer, N. Verma, E. Wu, and G. Strang, "A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level," *Proceedings of the National Academy of Sciences*, vol. 119, no. 32, p. e2123433119, 2022.
- [9] Wolfram Research. (2026) Linear systems. [Online]. Available: <https://reference.wolfram.com/language/guide/LinearSystems.html>